Jawaban Analisis UTS PBO

Nama : Torikh Abdullah Naser
NIM : 2410512031
Kelas : A-S1 Sistem Informasi

Analisis Soal 1

1. Mengapa Enkapsulasi penting pada sistem ini?

Enkapsulasi sangat krusial dalam sistem *e-learning* karena menyangkut **integritas data**. Jika properti seperti assignmentScore pada class Student dibuat public, maka siapa saja (bahkan dari luar sistem/class lain yang tidak berhak) dapat mengubah nilainya menjadi angka yang tidak masuk akal (contoh: diubah menjadi -50 atau 9999).
Dengan enkapsulasi (membuat field menjadi private dan menyediakan metode setter), sistem dapat menerapkan *validation rule*—seperti memblokir perubahan skor jika angkanya di luar jangkauan 0-100—sehingga data tetap akurat, konsisten, dan aman.

2. Mengapa Pewarisan (Inheritance) lebih baik dibanding duplikasi class?

Pewarisan memastikan sistem memenuhi prinsip **DRY (Don't Repeat Yourself)**. Baik Student maupun Instructor pada dasarnya adalah entitas User yang memerlukan properti id, name, dan fungsionalitas seperti login(). Jika tidak menggunakan pewarisan, kita harus menulis ulang variabel dan metode tersebut di setiap class secara terpisah.
Dengan pewarisan, kita hanya perlu menempatkan kode umum tersebut di User (Superclass), lalu menurunkannya. Ini membuat pemeliharaan kode (maintenance) lebih mudah, struktur lebih rapi, dan menambah tipe user baru (seperti Admin) ke depannya menjadi sangat cepat.

```
PS D:\kuliah\smt 4\prak pbo>  d:; cd 'd:\kuliah\smt 4\prak pbo'; & 'C:\Program Fi
ser\workspaceStorage\742f7bab6972091c9de5bfda8d2d49b3\redhat.java\jdt_ws\prak pbo
=== SISTEM E-LEARNING ===
[LOG] User Torikh (ID: STU-001) logged in at 2026-04-23 08:00:00
[LOG] User Dosen PBO (ID: INS-999) logged in at 2026-04-23 08:05:00
Instructor Dosen PBO memberikan nilai 85.5 kepada Torikh
Skor Akhir Torikh : 85.5
Error: Score harus antara 0 dan 100!
Total user terdaftar: 2
PS D:\kuliah\smt 4\prak pbo>
```

```java
package UTS_PBO;

class User {
    private final String id;
    private String name;
    private static int totalUsers = 0;

    protected class LoginActivity {
        private String timestamp;

        public LoginActivity(String timestamp) {
            this.timestamp = timestamp;
        }

        public void recordLog() {
            System.out.println("[LOG] User " + name + " (ID: " + id + ") logged in at " + timestamp);
        }
    }

    public User(String id, String name) {
        this.id = id;
        this.name = name;
        totalUsers++;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public static int getTotalUsers() { return totalUsers; }

    public void login(String timestamp) {
        LoginActivity activity = new LoginActivity(timestamp);
        activity.recordLog();
    }
}

class Student extends User {
    private double assignmentScore;

    public Student(String id, String name) {
        super(id, name);
        this.assignmentScore = 0.0;
    }

    public double getAssignmentScore() {
        return assignmentScore;
    }

    public void setAssignmentScore(double score) {
        if (score >= 0 && score <= 100) {
            this.assignmentScore = score;
        } else {
            System.out.println("Error: Score harus antara 0 dan 100!");
        }
    }

    public double calculateFinalScore() {
        return this.assignmentScore;
    }
}
```

```java
class Instructor extends User {
    public Instructor(String id, String name) {
        super(id, name);
    }

    public void gradeStudent(Student student, double score) {
        System.out.println("Instructor " + this.getName() + " memberikan nilai " + score + " kepada " + student.getName());
        student.setAssignmentScore(score);
    }
}

public class Soal1_ELearning {
    Run | Debug
    public static void main(String[] args) {
        System.out.println("=== SISTEM E-LEARNING ===");

        Student s1 = new Student("STU-001", "Torikh");
        Instructor i1 = new Instructor("INS-999", "Dosen PBO");

        s1.login("2026-04-23 08:00:00");
        i1.login("2026-04-23 08:05:00");

        i1.gradeStudent(s1, 85.5);

        System.out.println("Skor Akhir " + s1.getName() + " : " + s1.calculateFinalScore());

        s1.setAssignmentScore(150.0);

        System.out.println("Total user terdaftar: " + User.getTotalUsers());
    }
}
```

Analisis Soal 2

1. Tiga (3) Masalah Desain OOP pada Kode Awal & Dampaknya:

1. **Tidak Ada Enkapsulasi (Variabel Default/Package-Private):**
Variabel name dan score di class User tidak memiliki modifier private.
Dampak: Variabel bisa dimodifikasi dari luar class secara sewenang-wenang (contoh: student.score = 999;) sehingga logika atau batasan sistem hancur.

2. **Pelanggaran Batasan Superclass oleh Subclass:**
Pada class Student, method calculate() langsung mengakses score = score + 10;.
Dampak: Subclass menjadi terlalu bergantung pada implementasi internal superclass (*tight coupling*). Jika sewaktu-waktu cara penyimpanan data score di User berubah, class Student otomatis akan *error* (rusak).

3. **Tidak Memiliki Konstruktor dan Identifier Valid:**
Class tidak memaksa adanya data krusial saat objek diciptakan (konstruktor *default* kosong), dan tidak memiliki final ID.
Dampak: Objek bisa di-instansiasi tanpa nama dan nilai (menimbulkan NullPointerException). Ketiadaan ID yang unik dan statis (final) akan menyulitkan pelacakan spesifik saat data tersimpan di basis data nantinya.

2. Kapan Inner Class Lebih Tepat Digunakan Dibanding Class Terpisah?

Inner class (seperti ActivityLog atau LoginActivity) tepat digunakan jika class tersebut **hanya masuk akal atau hanya relevan jika berdiri di dalam konteks class utamanya (Outer Class)**.
Inner class juga bisa secara langsung mengakses variabel dan *method private* milik *Outer class* (seperti langsung membaca name dan id dari User tanpa perlu *getter* tambahan). Jika logika log aktivitas hanya relevan dan bergantung kuat pada eksistensi objek User tertentu, maka *Inner Class* jauh lebih *clean* secara arsitektur daripada membuat Class terpisah yang memaksa pertukaran data rumit.

3. Bandingkan: Desain Awal vs Desain Hasil Refactoring

Aspek	Desain Awal (Before)	Desain Refactoring (After)
Enkapsulasi	Tidak ada. Field public/default, bisa diubah bebas dari mana saja.	Semua field private. Akses hanya lewat getter/setter bervalidasi.
Keamanan Data	Rawan manipulasi. student.score = 999 valid secara sintaks.	Aman. setScore() menolak nilai di luar rentang 0-100.
Identitas Objek	Tidak ada ID. Sulit melacak objek spesifik.	Memiliki final id yang unik dan tidak bisa diubah setelah dibuat.

Aspek	Desain Awal (Before)	Desain Refactoring (After)
Keterlacakan	Tidak ada logging. Perubahan data tidak terekam.	Setiap perubahan data otomatis tercatat lewat *Inner Class* ActivityLog.
Scalability	Sangat rendah. Menambah fitur atau tipe user baru berisiko merusak kode lama (*fragile*).	Tinggi. Struktur *inheritance* yang benar dan akses via getter/setter memastikan subclass baru dapat ditambahkan tanpa merusak logika superclass.

Kesimpulan: Desain hasil refactoring jauh lebih *scalable*. Ini karena setiap class memiliki tanggung jawab yang jelas dan terisolasi (*Single Responsibility*). Perubahan pada satu bagian tidak berdampak berantai ke bagian lain, sehingga sistem siap berkembang tanpa mengorbankan stabilitas kode yang sudah ada.

```
PS D:\kuliah\smt 4\prak pbo>  & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.16.8-hotspo
1c9de5bfda8d2d49b3\redhat.java\jdt_ws\prak pbo_4b3866d2\bin' 'UTS_PBO.Soal2_Refactored'
=== SISTEM REFACTORING SOAL 2 ===
Torikh Abdullah Naser : 80
[SYSTEM LOG] Torikh Abdullah Naser (ID: 2410512031) -> Score diubah menjadi 90
Torikh Abdullah Naser : 90
PS D:\kuliah\smt 4\prak pbo>
```

```java
package UTS_PBO;

class UserRefactored {
    private final String id;
    private String name;
    private int score;

    protected class ActivityLog {
        public void log(String action) {
            System.out.println("[SYSTEM LOG] " + name + " (ID: " + id + ") -> " + action);
        }
    }

    public UserRefactored(String id, String name, int score) {
        this.id = id;
        this.name = name;
        this.score = score;
    }

    public String getId() { return id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getScore() { return score; }

    public void setScore(int score) {
        if (score >= 0 && score <= 100) {
            this.score = score;
            new ActivityLog().log("Score diubah menjadi " + score);
        } else {
            System.out.println("Error: Score harus antara 0 dan 100!");
        }
    }

    public void print() {
        System.out.println(name + " : " + score);
    }

    protected ActivityLog getLogger() {
        return new ActivityLog();
    }
}

class StudentRefactored extends UserRefactored {

    public StudentRefactored(String id, String name, int score) {
        super(id, name, score);
    }

    public void calculate() {
        int newScore = getScore() + 10;
        setScore(newScore);
    }
}

public class Soal2_Refactored {
    Run | Debug
    public static void main(String[] args) {
        System.out.println("=== SISTEM REFACTORING SOAL 2 ===");

        StudentRefactored student = new StudentRefactored("2410512031", "Torikh Abdullah Naser", 80);

        student.print();
        student.calculate();
        student.print();
    }
}
```